pontIA

APUNTES DE CLASE

Bases de Datos y SQL

8 de octubre de 2026

01 Tablas temporales vs CTE

- **Cuándo usar tabla temporal**
  Las tablas temporales son útiles para joins o selects rápidas con conjuntos de datos pequeños (por ejemplo, 5000 registros). Se crean, se usan y se destruyen.
- **Tablas grandes: no hacerlas temporales**
  Si la tabla tiene millones de registros, no se debe hacer temporal. Se genera una tabla con sus índices (por los campos del join o las agrupaciones), aunque sea temporal en concepto. Sin índices, la consulta se degrada mucho.

02 Motores SQL vs NoSQL

- **Principio ACID**
  Lo que diferencia un motor SQL de uno NoSQL es el cumplimiento del principio ACID. En un motor SQL la transacción siempre se cumple.
- **Casos de uso de cada motor**
  Los motores NoSQL se usan en compañías con grandes contenidos en internet, por su escalabilidad. Los motores SQL se orientan a controles transaccionales: sistemas bancarios, ERPs. Algunos CRMs están migrando a NoSQL porque no necesitan un nivel transaccional tan estricto, sobre todo los CRMs en modo SaaS basados en plataformas cloud.
- **Modelos híbridos**
  Existen modelos donde las transacciones se reciben en un motor SQL y luego se guardan en un motor NoSQL. Ejemplo: Amazon recibe las transacciones en un motor SQL (con réplicas) que asegura la transacción, y el resto va a un motor NoSQL.
- **Flexibilidad de esquema en NoSQL**
  En SQL las tablas tienen campos con estructura y tamaño definidos. En NoSQL el esquema es flexible: se puede hacer un insert con un campo adicional (por ejemplo, un campo 21 en una tabla de 20) sin que se rompa nada.
- **Distribución y almacenamiento**
  En un motor SQL la tabla está alojada en un espacio físico concreto. Las particiones en bases de datos relacionales son horizontales (por ejemplo, por fechas) y se pueden distribuir en distintos discos. En NoSQL, un objeto puede estar totalmente distribuido, lo que hace que la escritura, lectura y disponibilidad sean más rápidas.

PontIA

IDEA CLAVE

Si se necesita integridad transaccional, motor SQL; si no, motor NoSQL.

03 Particiones e índices

- **Particiones activas vs de solo lectura**
  Ejemplo: un libro contable particionado por trimestre. Si el cierre es trimestral, la partición del primer trimestre queda consolidada y solo tiene lecturas. Pero si el año fiscal está abierto completo y se añaden registros en cualquier trimestre, la partición es más rápida pero el índice se penaliza por las escrituras.
- **Evaluar la estrategia caso a caso**
  Hay que valorar: número de transacciones contra la tabla, si los cambios son severos o ligeros, y el número de lecturas. La optimización de bases de datos se tratará en la tercera clase. Es un tema frecuente en entrevistas de trabajo.

IDEA CLAVE

La estrategia entre particiones e índices depende del volumen de transacciones, el tipo de cambios y el número de lecturas contra la tabla.

PONTIA

04 Buenas prácticas en consultas SQL

- **Uso de alias**
  Conviene usar siempre alias en las tablas. Con una sola tabla no es imprescindible, pero cuando hay muchas tablas y varios joins, los alias permiten saber a qué tabla pertenece cada campo.
- **Legibilidad y formato de consultas**
  Es importante estructurar las consultas con los espacios e indentación adecuados para que sean legibles. La consulta funciona igual en una sola línea, pero bien formateada es mucho más fácil de leer. La mayoría de IDEs actuales ya formatean automáticamente.
- **Repaso de cláusulas básicas**
  `SELECT`, `FROM`, `WHERE`, `BETWEEN`, `LIKE`, `NULL`, funciones de agregación y filtros de agrupación ya se han visto en los recursos. Se pueden hacer consultas mucho más complejas: no hay que tener miedo a experimentar con la base de datos, ya que siempre se puede recuperar la copia de seguridad.

05 Propiedades ACID

- **Consistencia y restricciones**
  Las transacciones siempre deben respetar las restricciones de la base de datos. Si se intenta un `INSERT` en una tabla B sin que el registro referenciado exista en la tabla A, la transacción falla.
- **Aislamiento de transacciones**
  Dos transacciones concurrentes no se ejecutan realmente a la vez: el motor las serializa. Si ambas hacen `UPDATE` sobre el mismo registro, primero entra una y luego la otra. Para ello el motor bloquea el registro o incluso la tabla entera.
- **Commit y rollback**
  El `COMMIT` hace permanente la transaccion. El `ROLLBACK` la deshace y vuelve al estado anterior.

IDEA CLAVE

Los motores SQL relacionales garantizan consistencia fuerte mediante transacciones atómicas, aislamiento entre operaciones concurrentes, y persistencia con commit/rollback.

06 ON UPDATE CASCADE y ON DELETE CASCADE

- **Cuando usar CASCADE**
  Se pone cuando un cambio en la tabla A puede replicarse con seguridad en la tabla B sin riesgo de romper el sistema ni perder datos criticos.
- **Cuando no usar CASCADE**
  Cuando se quiere blindar un dato, no se pone CASCADE. Esto fuerza a hacer una doble comprobacion y los `UPDATE` correspondientes en ambas tablas, pero protege los datos criticos. En datos criticos no se suele poner.

IDEA CLAVE

No usar CASCADE en datos criticos: obliga a comprobar manualmente ambas tablas, pero protege contra perdida de datos.

07 Constraints

- **Foreign key (REFERENCES)**
  Referencia a otra tabla. Si se intenta un `INSERT` con un ID que no existe en la tabla referenciada, la operacion no se permite. En algunos motores como SQL Server se llama `FOREIGN KEY`; en otros se usa `REFERENCES`.
- **CHECK constraint**
  Restringe los valores de un campo a un conjunto definido, por ejemplo solo cuatro valores validos para un campo `status`. Antes habia que crear una tabla auxiliar con esos valores y relacionarla; ahora se define directamente a nivel de campo con `CHECK`. Es mas sencillo y funciona igual.

08 Vistas

- **Rendimiento de las vistas**
  Al ejecutar una vista, el motor genera primero todo el conjunto de datos y despues aplica el filtro. Si la vista devuelve millones de registros sin filtrar, penaliza mucho al motor. Una vista con prefiltrado estatico es mucho mas eficiente, pero ese filtrado no se puede cambiar dinamicamente.
- **Alternativas con parametros**
  Para filtrado dinamico se usan funciones o procedimientos almacenados, que permiten pasar parametros y filtrar la consulta en cada ejecucion. Las vistas son buena idea, pero hay que saber donde y cuando utilizarlas.

IDEA CLAVE

Las vistas son utiles para consultas repetidas con JOINs, pero hay que entender su impacto en rendimiento y cuando usar funciones o procedimientos almacenados en su lugar.

PontIA

09 Orden de ejecucion logico de SQL

- **Secuencia de ejecucion**
  `FROM` + `JOIN` → `WHERE` → `GROUP BY` + `HAVING` → `SELECT` → `DISTINCT` → `UNION` → `ORDER BY` → `LIMIT` + `OFFSET`.
- **Impacto del FROM y WHERE**
  Si el `FROM` con sus `JOIN` genera millones de registros y no hay un `WHERE` bien definido, se penaliza mucho al motor. Tener siempre un `WHERE` preciso es fundamental.
- **Evitar SELECT ***
  Con `SELECT *` el motor debe consultar una tabla de metadatos (en SQL Server se llama Nomenclator; PostgreSQL tiene algo similar) para resolver los campos. Con campos explicitos, el motor enlaza directamente tabla-campo-tipo sin ese paso extra. Ademas, `SELECT *` puede forzar un table scan en lugar de un index only scan, que es lo mas costoso para el motor. Solo usar `SELECT *` con un `LIMIT` pequeno para exploracion rapida.
- **Alias y filtrado**
  Usar alias siempre, especialmente con mas de una tabla. Filtrar siempre con `WHERE` cuando sea posible. Si aun no se sabe por que campo filtrar, usar `LIMIT` para sacar porciones pequenas de la tabla e ir explorando.
- **WHERE frente a HAVING**
  `WHERE` filtra filas antes de la agrupacion. `HAVING` filtra despues, sobre los resultados del `GROUP BY`.
- **UNION ALL frente a UNION**
  Preferir `UNION ALL` sobre `UNION` porque evita el ordenamiento y la eliminacion de duplicados. Si se necesitan datos ordenados y deduplicados y las tablas son pesadas, considerar lanzar la operacion como un batch o integrarla en un procedimiento almacenado que deje preparada la tabla.
- **ORDER BY solo al final**
  En procedimientos con tablas intermedias, aplicar `ORDER BY` solo a la tabla final, nunca a las intermedias. El `ORDER BY` es costoso, sobre todo sobre campos no indexados.
- **Indices B-tree para ordenacion**
  Si una tabla se ordena siempre por los mismos campos (por ejemplo fecha, status y un booleano en ese orden), conviene crear un indice B-tree especifico con esos campos en ese orden. El `ORDER BY` que siga ese mismo orden aprovecha el indice y es mucho mas rapido. Si se necesita un orden diferente, hay que generar un indice especifico para ello.
- **Evitar ORDER BY posicional**
  No usar numeros de posicion en el `ORDER BY` (por ejemplo `ORDER BY 3`). Obliga al motor a buscar en la query que campo corresponde a esa posicion en lugar de usar directamente el alias o nombre del campo.
- **ORDER BY por nombre, no por posición**
  Usar `ORDER BY 3` obliga al motor a buscar la posición del campo en la query, ir a la tabla a obtenerlo y luego ordenar. Con `ORDER BY nombre_campo` el motor resuelve directamente, sin ese trabajo extra.

PONIA

- **Formato e indentación del SQL**
  Cada cláusula en una línea diferente; en el `SELECT`, cada campo en su propia línea, todo indentado para que sea visualmente legible. Cuando el SQL va embebido en otro código (por ejemplo Python de un CRM), una consulta de 20 líneas sin formato es inmanejable. Bien formateada, cualquier cambio de campo, tabla o join se localiza al instante.

- **Modularizar con vistas o CTEs**
  Usar vistas, CTEs, procedimientos almacenados o funciones facilita el mantenimiento y evita tener que buscar queries dispersas en el código.

> IDEA CLAVE
> **El motor ejecuta primero FROM + JOIN y luego WHERE, por lo que un WHERE mal definido sobre un JOIN grande penaliza enormemente el rendimiento.**

> IDEA CLAVE
> **Indentar cada cláusula en una línea, usar nombres de campo en ORDER BY y modularizar con vistas o CTEs.**

10 Uso de IA en SQL

- **Entender lo que genera la IA**
  Se puede usar IA para generar queries, tablas o datos, pero hay que entender exactamente qué hace cada parte del código generado. Si no se entiende, hay un problema.

- **La IA no optimiza queries de verdad**
  La IA puede sugerir mejoras de estilo según estándares de PostgreSQL, Oracle, etc., pero no sabe qué índices tiene la tabla, si se está produciendo un index scan ni qué falta. Para eso existe el optimizador de consultas del motor de base de datos, que indica directamente dónde está el problema (por ejemplo, un índice que falta en el campo del join).

- **Optimizador de consultas vs IA**
  El resultado del optimizador se puede pasar a una IA, pero suele ser redundante porque el propio optimizador ya señala el problema. Con experiencia, se llega a revisar directamente el DDL de la tabla y detectar, por ejemplo, que no tiene índices por el campo del join.

- **IA para generar datos sintéticos**
  La IA es muy útil para generar tablas de ejemplo y datos sintéticos. Pero las tablas generadas probablemente no tendrán índices; al cargar un millón de registros y hacer joins, el rendimiento será malo y la optimización hay que hacerla manualmente.

> IDEA CLAVE
> **La IA ayuda a generar código y datos, pero la optimización real requiere conocer los índices, la estructura de las tablas y el optimizador de consultas.**

PontIA

11 Instalación de PostgreSQL y conexión

- **Alternativa con Docker**
  Si la instalación nativa de PostgreSQL da problemas (Mac, Linux), se puede instalar en un contenedor Docker. La única limitación es que los ejercicios de recuperación de ficheros pueden no funcionar igual, pero no es imprescindible. El restore de un dump se puede hacer con la CLI (`psql`).
- **Conexión en Linux con pgAdmin**
  En Linux, pgAdmin exige indicar una base de datos para conectar. Si esa base de datos no existe, falla. Solución: crear la base de datos desde la terminal antes de conectar, o usar la base de datos que viene por defecto en PostgreSQL (`postgres`) para la primera conexión.

12 Ejercicio práctico en grupo

- **SQL vs NoSQL en reservas aéreas**
  En un sistema crítico de reservas aéreas conviene un motor SQL relacional porque ACID garantiza transacciones seguras y consistentes: no se puede duplicar una misma reserva.
- **WHERE y HAVING**
  `WHERE` filtra antes de agrupar; `HAVING` filtra una vez agrupado.
- **JOIN por defecto es INNER**
  `JOIN` sin prefijo equivale a `INNER JOIN` .
- **Doble JOIN a la misma tabla**
  Cuando se necesitan el aeropuerto de salida y el de llegada, se hace JOIN a la tabla de aeropuertos dos veces, cada una con un alias distinto (p. ej. `AP` para salida y `AD` para llegada), cruzando por `airport_code` con `departure_airport` y `arrival_airport` de la tabla de rutas.
- **CTE frente a HAVING**
  Para filtrar resultados después de una agregación se puede usar `HAVING` tras el `GROUP BY` en lugar de crear una CTE.
- **Corrección de la consulta**
  La consulta del ejercicio combina `SELECT` con `COUNT` , varios `INNER JOIN` (desde `booking_routes`), filtro `WHERE` por vuelos cancelados, `GROUP BY` , `HAVING COUNT > 2` y `ORDER BY` descendente por total de cancelaciones.
- **GROUP BY completo**
  En el `GROUP BY` hay que incluir todas las columnas no agregadas del `SELECT` . Si falta alguna (por ejemplo el nombre de aeropuerto), la consulta falla o da resultados incorrectos.
- **COUNT y HAVING**
  `COUNT(*)` o `COUNT(fl.flight_id)` se coloca en el `SELECT` con un alias (por ejemplo `total_canceled`). El filtro `HAVING COUNT(...) > 2` va después del `GROUP BY` , no en el `WHERE` .
- **ORDER BY con alias**
  Para ordenar de mayor a menor cancelaciones se usa `ORDER BY total_canceled DESC` . Hay que indicar el campo por el que se ordena; no basta con poner solo `ORDER BY` .
- **Varios JOIN en una consulta**
  Se pueden encadenar varios `INNER JOIN` en la misma consulta. También es posible hacer un `JOIN` contra una subquery.

PontIA

IDEA CLAVE

Una consulta con varios INNER JOIN necesita GROUP BY con todas las columnas no agregadas, HAVING para filtrar por el agregado y ORDER BY para ordenar el resultado.

13 Creación de tabla con constraints

- **CHECK constraint en la tabla**
  La segunda parte del ejercicio pide crear una tabla con un `CHECK` constraint (igual que el visto antes con tipología) que limita los valores de una columna (por ejemplo severity) a `low`, `medium`, `high`.
- **FOREIGN KEY con ON DELETE CASCADE**
  La tabla incluye una `FOREIGN KEY` que referencia `flight_id` de otra tabla, con `ON DELETE CASCADE` para que al borrar el vuelo se eliminen también los registros dependientes.
- **INSERT y pruebas**
  Tras crear la tabla se hace un `INSERT`. Se recomienda probar con valores válidos e inválidos: poner un `flight_id` que no exista o un valor de severity fuera de los permitidos.
- **Violación del constraint**
  Al insertar un valor que no cumple el `CHECK` constraint, la base de datos devuelve el error `violates check constraint` y rechaza la inserción.
- **Sintaxis del constraint con CHECK**
  La palabra `CONSTRAINT` va delante del nombre del check. La definición del `CHECK` se pone en la misma línea que la columna. Hay que cerrar un paréntesis para el `CHECK` y otro para el `CREATE TABLE`.

IDEA CLAVE

El CHECK constraint restringe los valores permitidos en una columna; si se intenta insertar un valor fuera del rango definido, la base de datos devuelve un error.

14 Claves foráneas y vistas

- **REFERENCES exige tabla**
  Una foreign key tiene que referenciar siempre una tabla, no una vista. La tabla es la que tiene un identificador asociado a una clave en la metadata. Una vista contiene campos, pero no tiene claves primarias ni foráneas.
- **Nombre de columna en el enunciado vs. en la base de datos**
  La descripción de una columna puede poner un nombre (por ejemplo `aircraft_code`) que no coincide con el nombre real de la columna en la tabla (`airplane_code`). Hay que verificar siempre el nombre real antes de escribir el `REFERENCES`.

IDEA CLAVE

Una clave foránea (REFERENCES) solo puede apuntar a una tabla, nunca a una vista.

PontIA

15 Consulta con JOINs, agrupación y filtrado

- **FILTER y CASE como alternativas**
  Para obtener conteos condicionales (por ejemplo, vuelos cancelados) dentro de un `SELECT` con `GROUP BY`, se puede usar `COUNT(*) FILTER (WHERE condicion)` o bien una estructura `CASE WHEN... THEN... END` dentro del `COUNT`. Ambas formas son válidas.

- **Estructura de la consulta**
  `SELECT` con código y modelo de avión, `COUNT(*)` para total de vuelos y conteo filtrado para vuelos cancelados. `INNER JOIN` entre tablas de vuelos, rutas y aviones. `GROUP BY` por código y modelo. `HAVING COUNT(*) > 5` para quedarse solo con aeronaves con más de 5 vuelos asignados. `ORDER BY` vuelos cancelados de mayor a menor.

- **Inspeccionar tablas antes de hacer JOINs**
  Cuando un JOIN falla, conviene hacer `SELECT * FROM tabla LIMIT 10` de cada tabla implicada para ver sus columnas y entender por dónde se deben unir.

- **GROUP BY completo**
  Si el `SELECT` incluye columnas no agregadas (por ejemplo código y modelo), todas deben aparecer en el `GROUP BY`. Si falta alguna, la consulta da error.

> IDEA CLAVE
> Para contar con condición dentro de un SELECT agrupado se puede usar `COUNT(*) FILTER (WHERE...)` o una estructura `CASE`.

16 INSERT y DESCRIBE

- **Preparar un INSERT**
  Antes de escribir un `INSERT INTO`, conviene hacer un `DESCRIBE` de la tabla destino para ver sus columnas y tipos. Así se evitan errores al redactar la sentencia.

17 Desactivar IntelliSense en el editor SQL

- **Opciones para desactivar autocompletado**
  Si el autocompletado (Copilot o IntelliSense) interfiere al escribir SQL: 1) Ir a extensiones y desactivar Copilot. 2) En Settings buscar "IntelliSense" y desactivar el IntelliSense de SQL. 3) En el editor, pulsar los tres puntos a la derecha del botón de ejecutar → Configurar editores → eliminar Copilot de la lista. 4) En la configuración de SQL Tools también se puede desactivar.

18 Ejercicio práctico: INSERT, UPDATE y DELETE

- **Inserción con referencia a otra tabla**
  Para insertar registros con una foreign key, primero se consulta la tabla referenciada (`SELECT * FROM airplanes_data LIMIT 10`) para obtener códigos válidos. Se insertan varias filas con distintos estados: `in_progress`, `scheduled` y `cancelled`.

PontIA

- **UPDATE con condición**
  El `UPDATE` se aplica solo a los registros que cumplen una condición concreta en el `WHERE`, no a toda la tabla.
- **DELETE de registros cancelados**
  Se ejecuta un `DELETE` filtrando por el estado `cancelled`. Por eso se insertan previamente registros con ese estado: para poder practicar el borrado.

19 Las vistas no admiten foreign keys

- **Metadata solo para tablas**
  PostgreSQL mantiene metadata implícita para cada objeto, incluidas las vistas. Sin embargo, la metadata relativa a primary keys, foreign keys y constraints solo está habilitada para tablas. Al intentar crear una referencia (foreign key) contra una vista, PostgreSQL no encuentra la metadata necesaria y devuelve error.

> IDEA CLAVE
> **Las constraints (primary key, foreign key) solo están habilitadas para tablas, no para vistas.**

20 SQL vs NoSQL para telemetría de sensores

- **Esquema variable según fabricante**
  La telemetría de un avión (presión, temperatura, datos binarios cada segundo) varía en número de parámetros según el fabricante e incluso según el motor. Un fabricante puede enviar 4 datos y otro 7. En una base de datos SQL habría que tener definidas todas las columnas posibles para cada tipo de motor; si llega uno nuevo con más parámetros, hay un problema. Una base de datos NoSQL maneja mejor esa variabilidad de esquema.

> IDEA CLAVE
> **La telemetría de sensores con esquema variable entre fabricantes encaja mejor en una base de datos NoSQL.**

21 Uso de IA para generar SQL

- **Recomendación práctica**
  Se puede extraer el esquema completo de las tablas (en imagen o en código), pasárselo a una IA y pedirle que genere el SQL. No hay problema en hacerlo; en el día a día profesional con bases de datos es lo habitual. Por ejemplo, preguntar a la IA cómo crear un campo autonumérico en PostgreSQL.
- **Entender el código generado**
  La IA debe ser una herramienta de ayuda, no un sustituto del pensamiento propio. Cuando genere código, hay que entender bien qué está haciendo.

PontIA

22 Entrega de ejercicios

- **Requisito para la práctica final**
  Cada módulo tiene ejercicios o cuestionarios que se deben entregar. Es obligatorio completarlos todos antes de entregar la práctica final, ya que sin ellos no se puede corregir. No hay que tener miedo a equivocarse: los errores forman parte del aprendizaje.

PontIA